# CS 423
# Operating System Design: Persistence: RAID and FS Design
# Nov 5

Ram Alagappan

# AGENDA / LEARNING OUTCOMES

Finish RAID -

- Levels 4 and 5
- performance characteristics

Going up the stack – from devices to file systems

# RECAP

# ONLY ONE DISK?

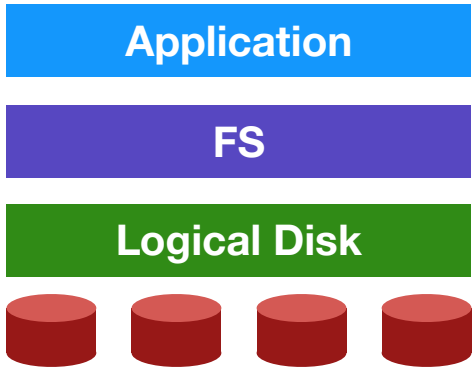
Sometimes we want many disks — why?

- capacity
- reliability
- performance

Challenge: most file systems work on only one disk

# RAID

Build logical disk from many physical disks.

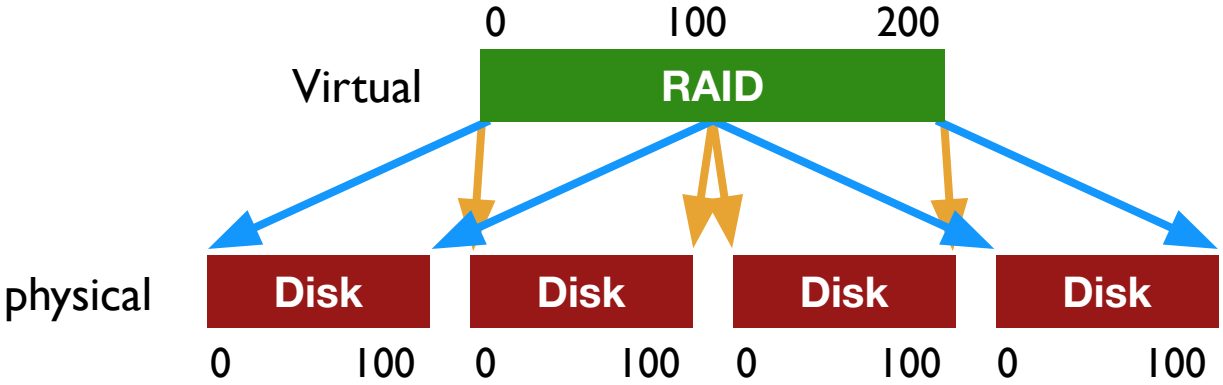



Logical disk gives capacity, performance, reliability

RAID: **R**edundant **A**rray of **I**nexpensive **D**isks

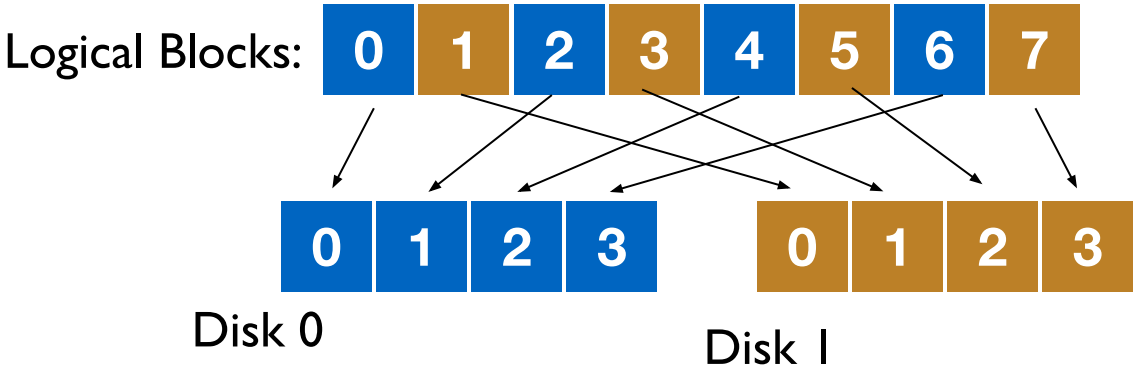
Transparency: no changes to the FS, host, Apps → ease of deployment (really important for adoption!)

# GENERAL STRATEGY: MAPPING, REDUNDANCY

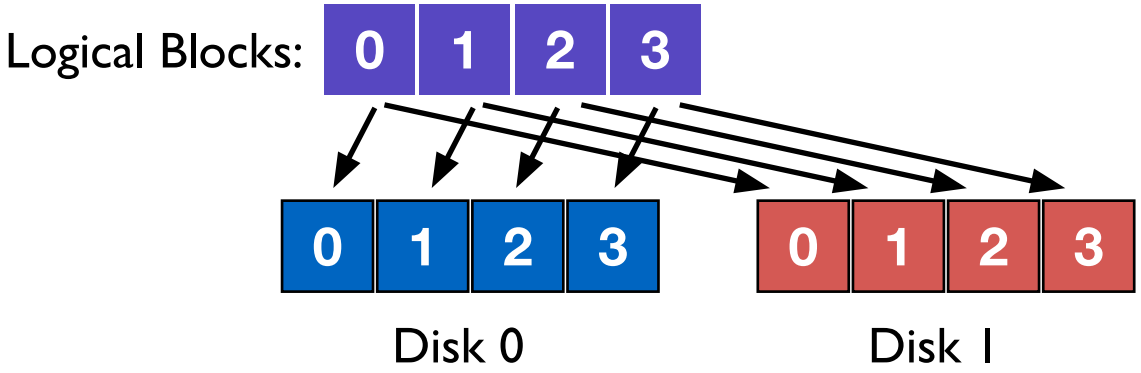


# RAID-0: STRIPING

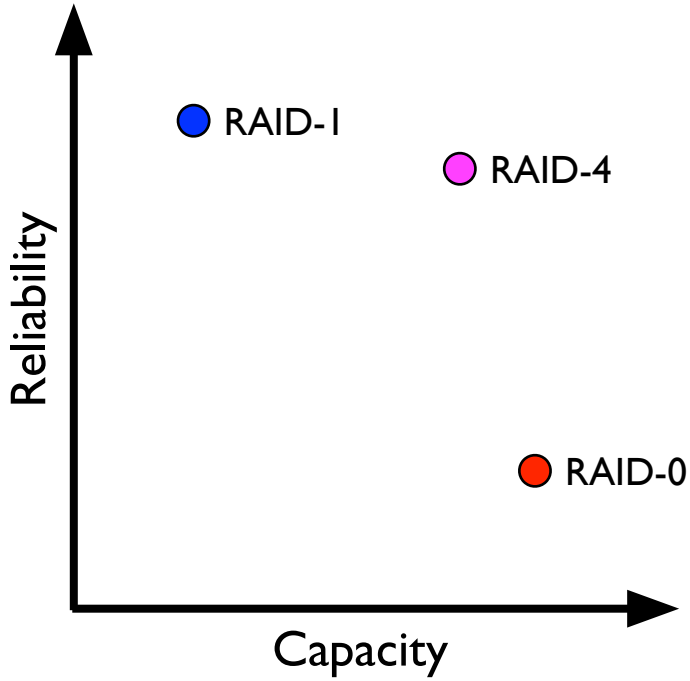
Optimize for capacity. No redundancy



# RAID-1: MIRRORING




Keep two copies of all data.


RAID-1
RAID-4
RAID-0
Reliability
Capacity

# END RECAP

# RAID-4 STRATEGY

Use **parity** disk

If an equation has N variables, and N-1 are known,
you can solve for the unknown.

Treat sectors across disks in a stripe as an equation.

Data on bad disk is like an unknown in the equation.

# RAID 4: EXAMPLE

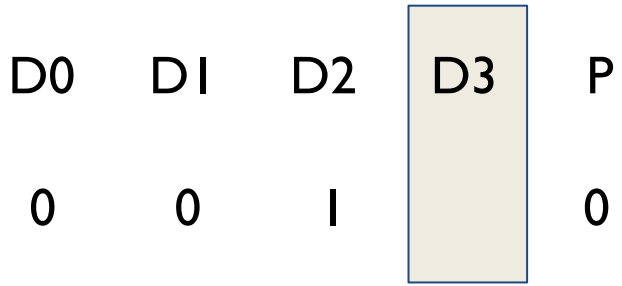
| Disk 0 | Disk 1 | Disk 2 | Disk 3 | Disk 4 |
|---|---|---|---|---|
| 0 | 1 | 2 | 3 | P0 |
| 4 | 5 | 6 | 7 | P1 |
| 8 | 9 | 10 | 11 | P2 |
| 12 | 13 | 14 | 15 | P3 |

What functions can we use to compute parity?

| C0 | C1 | C2 | C3 | P |
|---|---|---|---|---|
| 0 | 0 | 1 | 1 | XOR(0,0,1,1) = 0 |
| 0 | 1 | 0 | 0 | XOR(0,1,0,0) = 1 |

# RAID 4: EXAMPLE

Let's say disk D3 is down. How do we read its contents?



# UPDATES: ADDITIVE VS SUBTRACTIVE PARITY

| C0 | C1 | C2 | C3 | P0 |
|---|---|---|---|---|
| 0 | 0 | 1 | 1 | XOR(0,0,1,1) |

Additive Parity:
read all old blocks, recalculate parity block value

Subtractive Parity
Idea:
if old and new same, then xor
parity would not change, so keep it as is

if new data bit and old data bit are different, flip the parity

$$P_{new} = (C_{old} \oplus C_{new}) \oplus P_{old}$$

# RAID-4: ANALYSIS

What is the capacity?

How many disks can fail?

Latency (read, write)?

N := number of disks
C := capacity of 1 disk
S := sequential throughput of 1 disk
R := random throughput of 1 disk
D := latency of one small I/O operation

# RAID-4: THROUGHPUT

Assume subtractive parity. What is steady-state throughput for

- sequential reads?
- sequential writes?
- random reads?
- random writes? (next page!)

# RAID-4: RANDOM WRITE PERFORMANCE

| C0 | C1 | C2 | C3 | P0 |
|---|---|---|---|---|
| 0 | 0 | 1 | 1 | XOR(0,0,1,1) |

Subtractive Parity

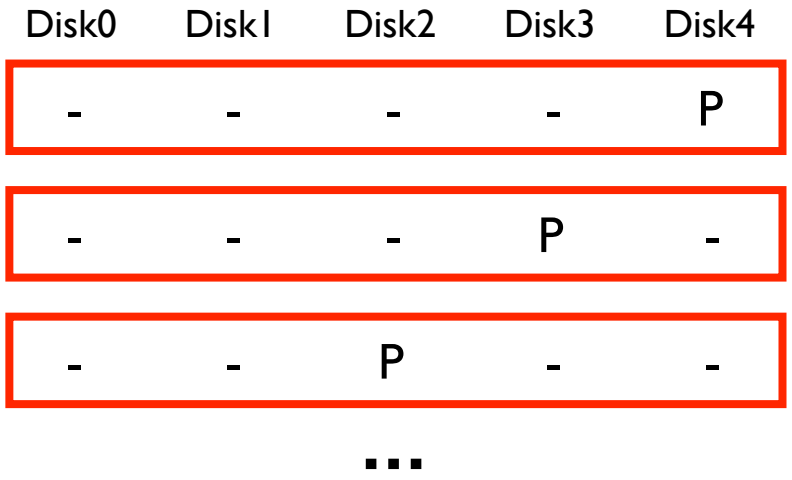
$$P_{new} = (C_{old} \oplus C_{new}) \oplus P_{old}$$

Assume subtractive parity

Think which disk is the bottleneck here with many random IOs? By how much?

| Disk 0 | Disk 1 | Disk 2 | Disk 3 | Disk 4 |
|---|---|---|---|---|
| 0 | 1 | 2 | 3 | P0 |
| *4 | 5 | 6 | 7 | +P1 |
| 8 | 9 | 10 | 11 | P2 |
| 12 | *13 | 14 | 15 | +P3 |

# RAID-5




Rotate parity across different disks
A single disk is not responsible for parity anymore

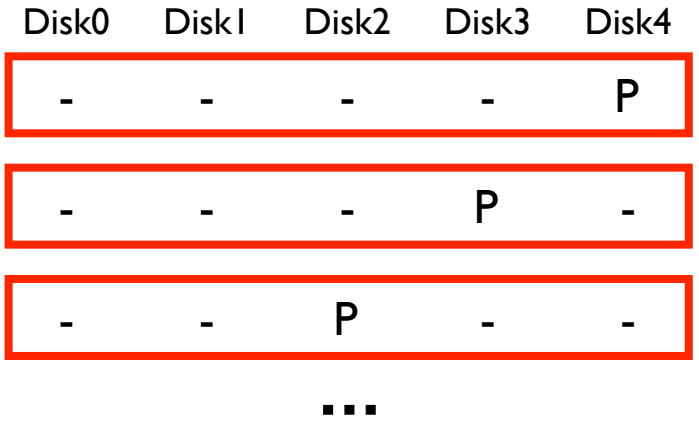
# RAID-5: ANALYSIS

What is the capacity?

How many disks can fail?

Latency (read, write)?

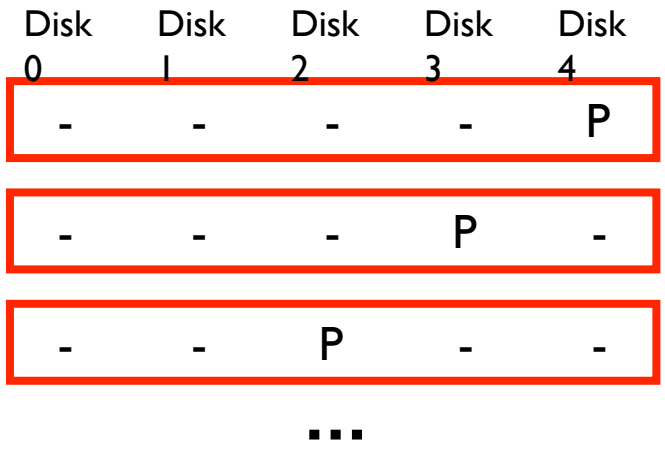
N := number of disks
C := capacity of 1 disk
S := sequential throughput of 1 disk
R := random throughput of 1 disk
D := latency of one small I/O operation



# RAID-5: THROUGHPUT

What is steady-state throughput for RAID-5?

- sequential reads?
- sequential writes?
- random reads?
- random writes? (next page!)



# RAID-5 RANDOM WRITES

| Disk 0 | Disk 1 | Disk 2 | Disk 3 | Disk 4 |
|---|---|---|---|---|
| 0 | 1 | 2 | 3 | P0 |
| 5 | 6 | 7 | P1 | 4 |
| 10 | 11 | P2 | 8 | 9 |
| 15 | P3 | 12 | 13 | 14 |
| P4 | 16 | 17 | 18 | 19 |

# RAID IN THE REAL WORLD

Many modern SSDs internally use RAID-like parity across multiple NAND dies/channels – called RAIN – redundant array of ind. NANDs.

e.g., 8 NAND components might be organized as 7 data + 1 parity, RAID-5-like, but the exact layout varies by SSD

Enterprise servers / storage arrays (e.g., Dell poweredge servers)

NAS – network attached storage appliances (e.g., Synology DiskStation)

# RAID IN DISTRIBUTED SYSTEMS?

RAID-1 mirrors data across two disks

Conceptually similar to data replication across servers in distributed systems (e.g., Primary-backup servers, Paxos/Raft-based systems)

Raid-4 and 5 store parity instead of full replicas

This is a simple form of **erasure coding**

Distributed systems generalize this idea using codes like Reed–Solomon, which can tolerate multiple failures with less storage overhead than full replication.

# RAID LEVEL COMPARISONS

| | Reliability | Capacity | Read latency | Write Latency | Seq Read | Seq Write | Rand Read | Rand Write |
|---|---|---|---|---|---|---|---|---|
| RAID-0 | 0 | C*N | D | D | N * S | N * S | N * R | N * R |
| RAID-1 | 1 | C*N/2 | D | D | N * S | N/2 * S | N * R | N/2 * R |
| RAID-4 | 1 | (N-1) * C | D | 2D | (N-1)*S | (N-1)*S | (N-1)*R | R/2 |
| RAID-5 | 1 | (N-1) * C | D | 2D | (N-1)*S | (N-1)*S | N * R | N/4 * R |

# RAID QUIZ

RAID-4 and RAID-5 both have a single-device hotspot (i.e., the parity) in terms of performance. T or F?

In an 5-disk RAID-5 array, approximately 10% of the blocks on each disk store parity. T or F?

In an N-disk RAID-5 array, a small random write to one data block requires updating exactly two disks, regardless of N. T or F?

A 3-disk RAID-5 array improves usable capacity over 2-disk RAID-1. T or F?

If True, by how much?

# DISKS □ FILES

# GOING UP THE STACK...

# PLAN

- This lecture:
  - high-level overview of FS elements
  - design of a simple file system
  - introduce the problem of crash consistency

- Next: solutions for crash consistency

# FILES

Array of persistent bytes that can be read/written

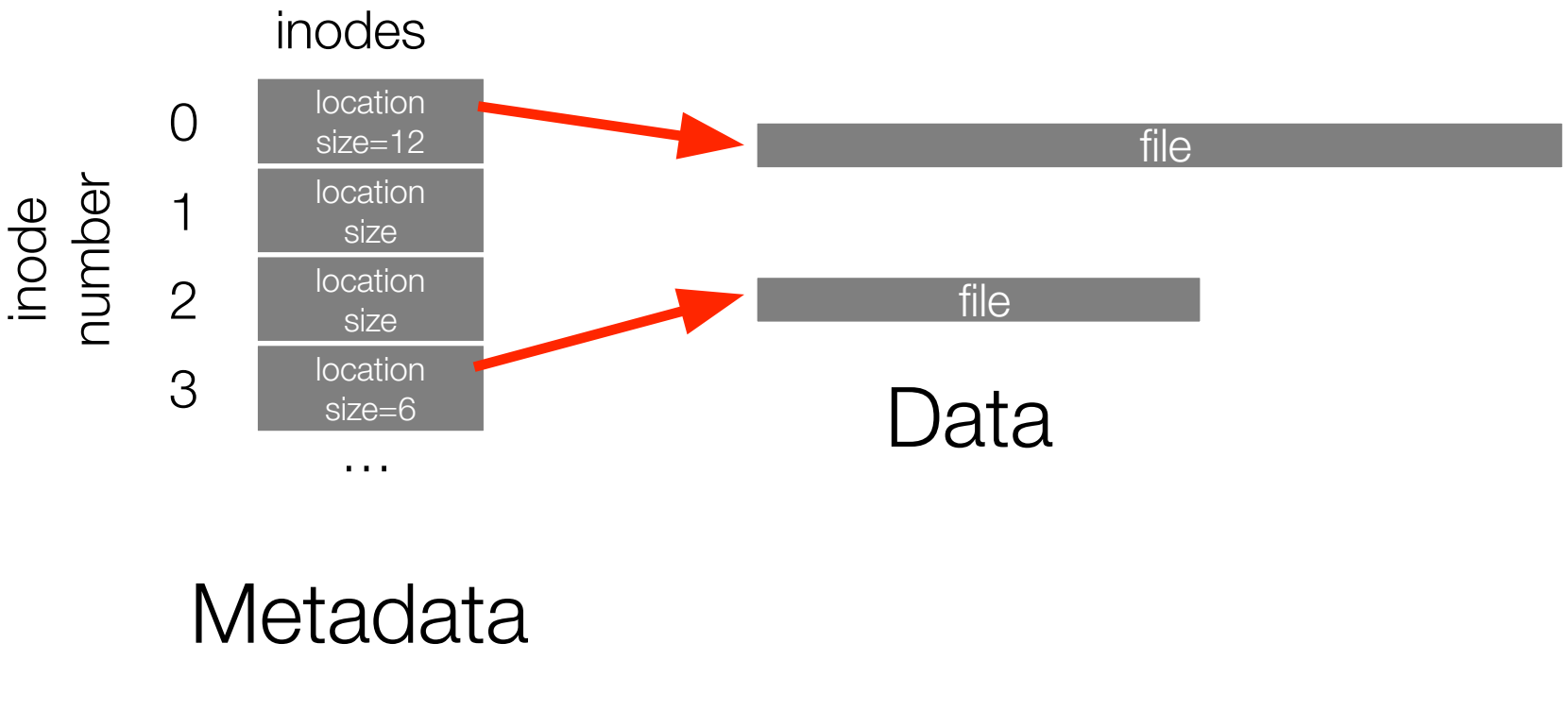
**File system** consists of many files

the part of OS that manages those files

Files need names to access correct one

Usually, two types of names

- inode numbers (low-level)
- Pathname (easier for humans)


inodes
inode number
0
1
2
3
location size=12
location size
location size
location size=6
...
file
file
Data
Metadata

# PATHS AND DIRECTORIES

String names are friendlier than number names

File system internally interacts with inode numbers

Store *path-to-inode* mappings in a special file or rather a *Directory*!

# DIRECTORY

A directory is simply a special file with its data in a specific format

A directory have many files and also directories in turn, each identified by a name

So, a directory contains a list of string names pointing to inode numbers

Note that some inodes may be directories here…

| | |
|---|---|
| foo | 41 |
| bar | 02 |
| abc | 35 |
| xyz.txt | 88 |

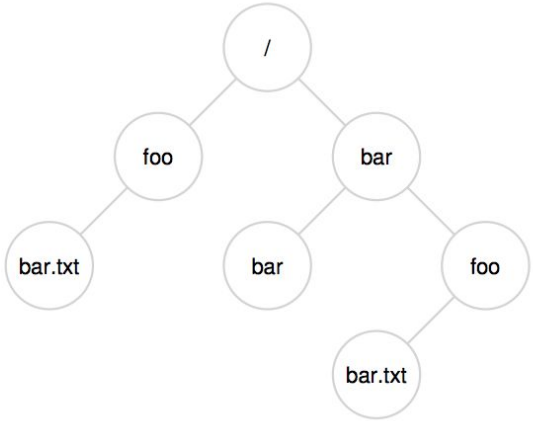
# HIERARCHICAL DIRECTORIES

Directory Tree instead of single root directory

File name needs to be unique within a directory

/usr/lib/file.so

/tmp/file.so




Store file-to-inode mapping in each directory

# FS DESIGN

Very Simple File System (VSFS)

Two aspects:

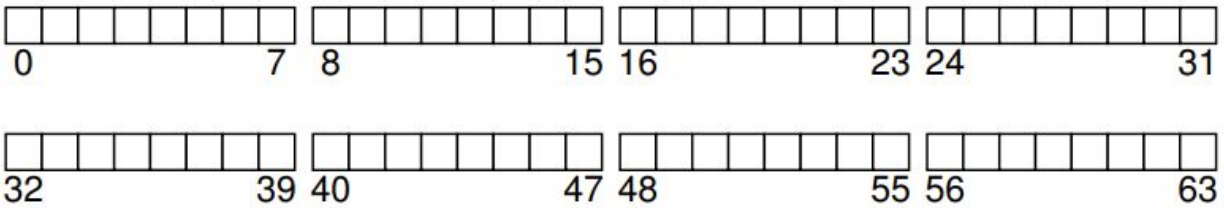
- Data structures – how are files, directories, etc stored on disk
- Access methods – how are high-level operations like open, read, write mapped to these DS operations

Relevance to the real world:

- VSFS resembles ext2, a Linux FS - precursor of ext3/4

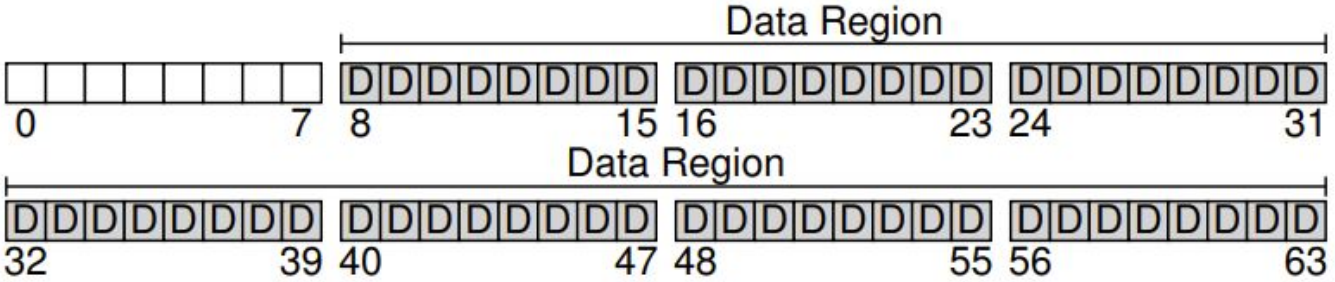
# ON-DISK LAYOUT

Assume a small disk partition with 64 blocks

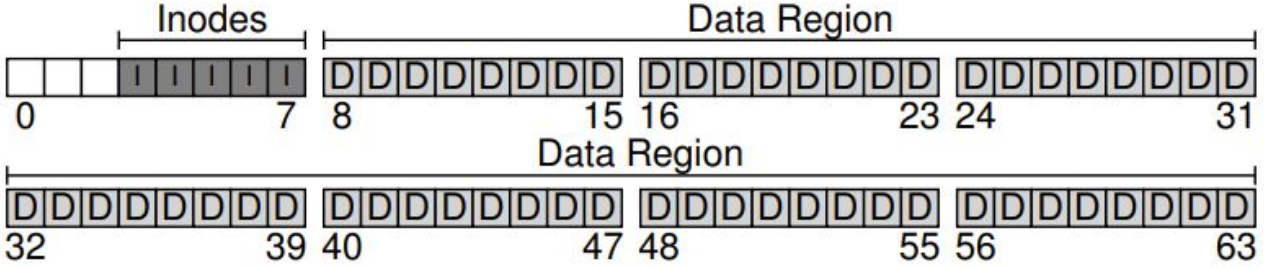



Data and metadata – most space must go for data blocks

# DATA BLOCKS



# VSFS – Inodes (metadata)




Called the inode table

With 256-byte inodes, we can store _____ inodes in a block, so totally _____ files can be stored in VSFS

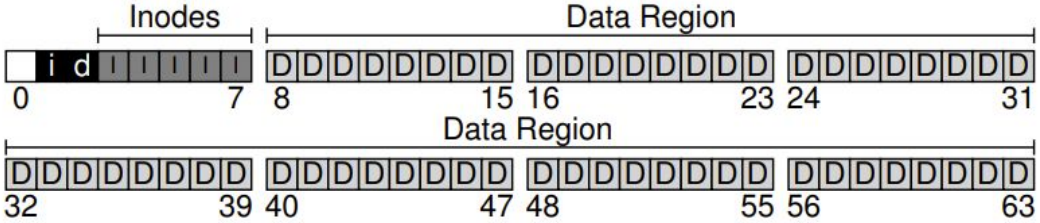
But we can simply scale VSFS to a larger disk

# VSFS – Bitmaps (metadata)

Need allocation structures

Free lists – linked list is an option

Most commonly used: bitmap (ib, db)




ib – inode bitmap - Nth bit indicates whether inode N is free or not

ib – data bitmap - Nth bit indicates whether data block N is free or not

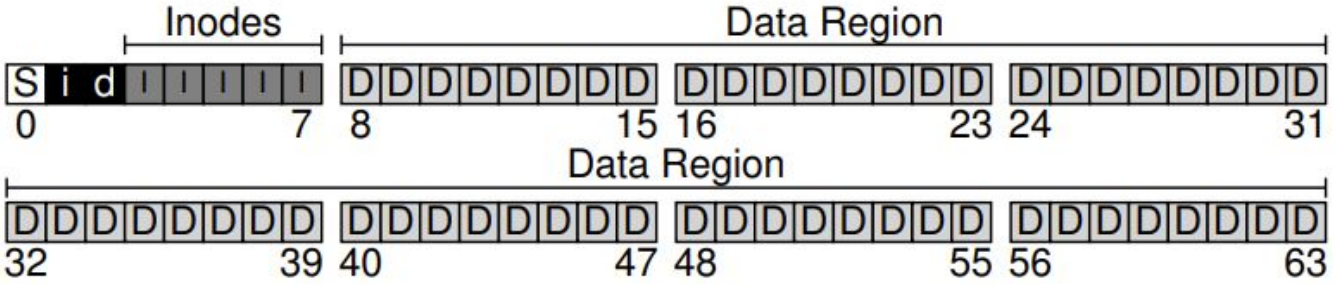
Note: in this specific FS instance, we don't need a full block for the bitmap

How much space do you actually need for data bitmap? inode bitmap?

# VSFS – one more metadata block

What's stored in the first block?

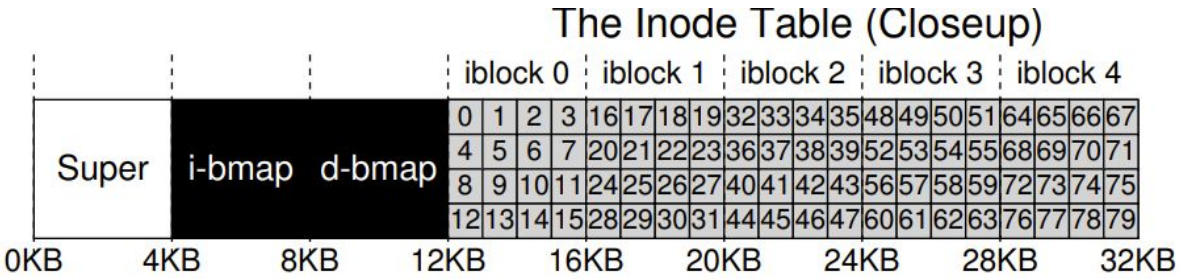
# VSFS – Superblock (metadata)




How many data blocks, how many inode blocks

Inode table starting block #

# INODE




Implicitly know the block + offset from the inode number
inode 16 is at offset 0 of iblock1 (1st block of inode table)

# WHAT DOES AN INODE CONTAIN?

| Size | Name | What is this inode field for? |
|---|---|---|
| 2 | mode | can this file be read/written/executed? |
| 2 | uid | who owns this file? |
| 4 | size | how many bytes are in this file? |
| 4 | time | what time was this file last accessed? |
| 4 | ctime | what time was this file created? |
| 4 | mtime | what time was this file last modified? |
| 4 | dtime | what time was this inode deleted? |
| 2 | gid | which group does this file belong to? |
| 2 | links_count | how many hard links are there to this file? |
| 4 | blocks | how many blocks have been allocated to this file? |
| 4 | flags | how should ext2 use this inode? |
| 4 | osd1 | an OS-dependent field |
| 60 | block | a set of disk pointers (15 total) |
| 4 | generation | file version (used by NFS) |
| 4 | file_acl | a new permissions model beyond mode bits |
| 4 | dir_acl | called access control lists |

A few interesting fields:

size

mtime

links_count

block – 15 4-byte pointers

# LINK AND UNLINK

An inode can be pointed to by many names!

links_count in inode = # of such links to the inode

Link syscall in Linux

Delete a filename: which syscall in Linux?

Delete and free the data blocks of inode when no name points to it

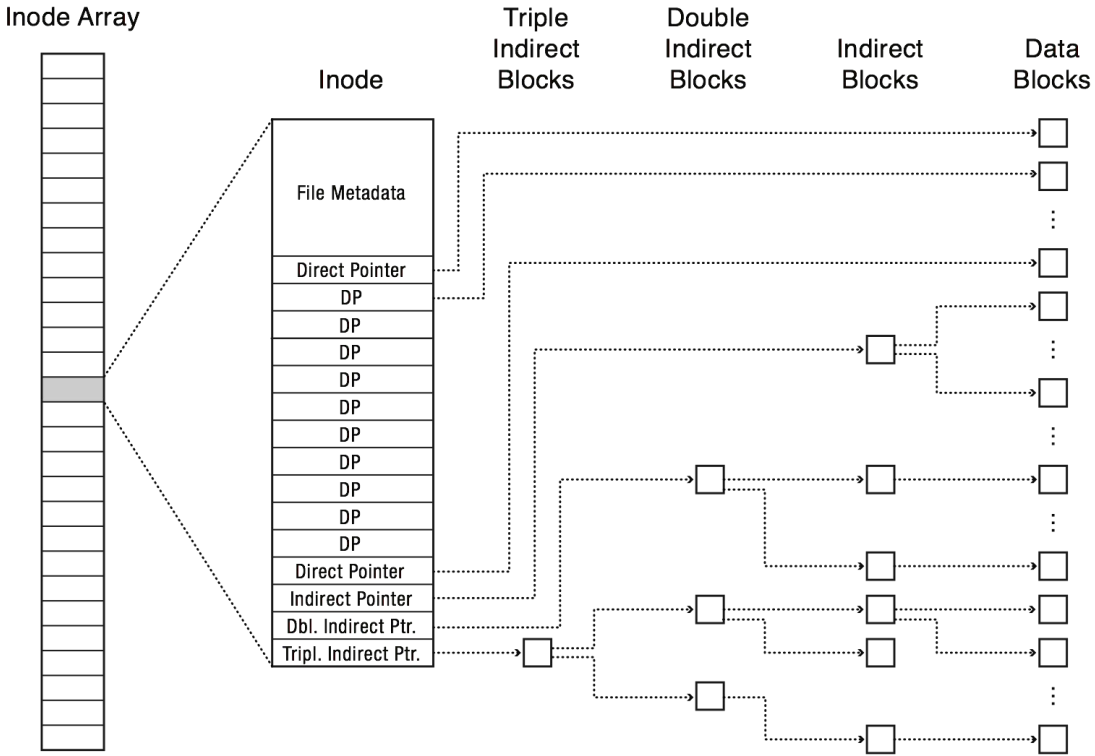
# WHAT IS THE MAX FILE SIZE?

We have 15 block pointers

What is the max file size?

How can we support larger files?

# DIRECT AND INDIRECT POINTERS



# FILE SIZE

File size with one indirect pointer + 12 direct:

1024 * 4K + 12*4K  – roughly 4MB

File size with 1 double ID + 1 ID + 12 direct:

1024 * 1024 * 4K + 1024 * 4K + 12*4K – roughly 4GB

# Small files: Inlined

- Really small files
- No need to have a separate data block
- Inline data into the inode – can access with fewer disk accesses

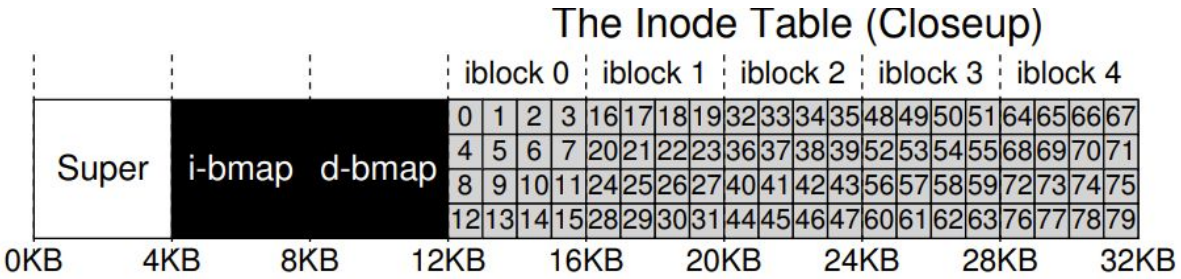
# DIRECTORY ORGANIZATION

| inum | reclen | strlen | name |
|---|---|---|---|
| 5 | 12 | 2 | . |
| 2 | 12 | 3 | .. |
| 12 | 12 | 4 | foo |
| 13 | 12 | 4 | bar |
| 24 | 36 | 28 | foobar_is_a_pretty_longname |

What is the inode of this directory?

What is the inode of its parent?

# DIRECTORY ORGANIZATION




Where is a directory's content stored?

No special block type for it !?

# CREATING AND WRITING FILE (TIME GOES DOWNWARDS)

| | data bitmap | inode bitmap | root inode | foo inode | bar inode | root data | foo data | bar data [0] | bar data [1] |
|---|---|---|---|---|---|---|---|---|---|
| create (/foo/bar) | B | read write | read | read | read write D write | read | read A | write C | |
| write() | read write | | | | read E write | | | write | |
| write() | read write | | | | read write | | | | write |

Why read foo data? A

Why read inode bitmap? B

What's written in foo data? C

Why write foo inode? D

Why write bar inode on data write? E

# CREATING AND WRITING FILE (TIME GOES DOWNWARDS)

| | data bitmap | inode bitmap | root inode | foo inode | bar inode | root data | foo data | bar data [0] | bar data [1] |
|---|---|---|---|---|---|---|---|---|---|
| create (/foo/bar) | B | read write | read | read | read write D write | read | read A | write C | |
| write() | read write | | | | read write E | | | write | |
| write() | read write | | | | read write | | | | write |

How do we know the root's inode number to bootstrap !?

# PAGE CACHE

Disk access is expensive

Can cache blocks in memory – all FS do this

Integrated with virtual memory

    can balance fs cache vs. vm

Also helps write buffering

Flushing deamon

# CRASH CONSISTENCY

Basic problem:

Must update many data structure on disk as a unit

What if failure happens in the middle

Types of failure:

- kernel panic
- power failures

Remember the consistent-update problem in RAID-1?

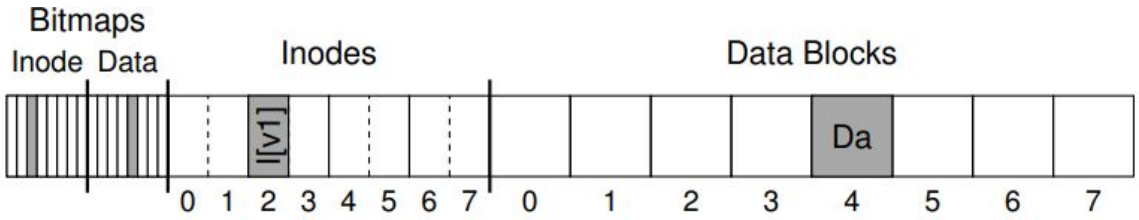
# CREATE FILE AND APPEND A BLOCK EXAMPLE

Let's say we create a file and append a block to it…

How many blocks do we need to write to accomplish the append?

Which ones?

# CREATE AND APPEND A BLOCK EXAMPLE



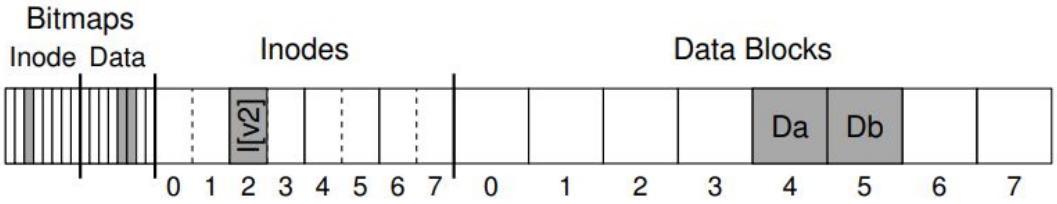

inode bitmap – allocate the inode for the created file

inode bitmap – allocate the inode for the created file

Data block – the actual data block

inode – modify the inode (change size, add pointer)

# PROBLEMS




What if only Db is written?
Only i[V2] is written to disk? (2 problems)
Data bitmap is alone written to disk?
Bitmap and data are written:
Data and inode are written:
Bitmap and inode are written:

What's special about the last case?

# METADATA VS. DATA

FS Metadata consistency vs. Data consistency

FS metadata consistency: internal structures agree with each other

Data consistency: additionally, the data must “make sense” to applications and users

# NEXT LECTURE

How to ensure FS crash consistency

What techniques: FSCK, journaling

Log-structured  file systems